

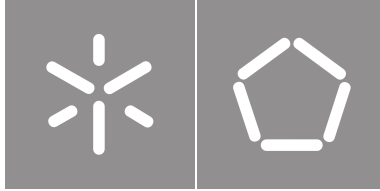


Rui Pedro Carvalho Dias

**Estratégias de sincronização de dados em
redes veiculares NDN.**

Universidade do Minho
Escola de Engenharia





Universidade do Minho

Escola de Engenharia

Rui Pedro Carvalho Dias

**Estratégias de sincronização de dados
em redes veiculares NDN.**

Dissertação de Mestrado

Trabalho efetuado sob a orientação de:

**Maria João Mesquita Rodrigues Cunha Nicolau
Pinto**

António Luís Duarte Costa

DIREITOS DE AUTOR E CONDIÇÕES DE UTILIZAÇÃO DO TRABALHO POR TERCEIROS

Este é um trabalho académico que pode ser utilizado por terceiros desde que respeitadas as regras e boas práticas internacionalmente aceites, no que concerne aos direitos de autor e direitos conexos.

Assim, o presente trabalho pode ser utilizado nos termos previstos na licença abaixo indicada.

Caso o utilizador necessite de permissão para poder fazer um uso do trabalho em condições não previstas no licenciamento indicado, deverá contactar o autor, através do RepositoriUM da Universidade do Minho.

Licença concedida aos utilizadores deste trabalho



Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International
CC BY-NC-SA 4.0

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.en>

Agradecimentos

DECLARAÇÃO DE INTEGRIDADE

Declaro ter atuado com integridade na elaboração do presente trabalho académico e confirmo que não recorri à prática de plágio nem a qualquer forma de utilização indevida ou falsificação de informações ou resultados em nenhuma das etapas conducente à sua elaboração.

Mais declaro que conheço e que respeitei o Código de Conduta Ética da Universidade do Minho.

_____, _____
(Local) (Data)

(Rui Pedro Carvalho Dias)

Resumo

Estratégias de sincronização de dados em redes veiculares NDN.

Relativamente ao seu conteúdo, os resumos não devem ultrapassar uma página e frequentemente tentam responder às seguintes questões (é imprescindível a adaptação às práticas habituais da sua área científica):

1. Qual é o problema?
2. Porque é que é um problema interessante/desafiante?
3. Qual é a proposta de abordagem/solução?
4. Quais são as consequências/resultados da solução proposta?

Palavras-chave: Primeira palavra-chave, Outra palavra-chave, Mais uma palavra-chave, A última palavra-chave

Índice

Índice de Figuras	vii
Índice de Tabelas	viii
Siglas	x
1 Introdução	1
1.1 Enquadramento e Motivação	1
1.2 Objetivos	1
2 Estado da Arte	2
2.1 Redes Veiculares	2
2.1.1 Desafios das redes veiculares	3
2.2 Redes NDN	3
2.2.1 Funcionamento do Named Data Network (NDN)	4
2.3 Vehicular Named Data Network (Redes veiculares NDN (V-NDN))	5
2.3.1 Trabalhos em (V-NDN)	6
2.4 Sincronização no NDN	6
2.4.1 State Vector Sync State Vector Sync (SVSync)	8
2.4.2 Desafios das Redes Veiculares	11
3 Estratégia a utilizar	12
4 Implementação	14
5 Resultados	16
6 Conclusão e trabalho futuro	20
Bibliografia	21

Índice de Figuras

1	Mensagens das Redes veiculares ad hoc (VANETs) [1]	2
2	Aplicações das VANETs [4]	3
3	Pacotes das redes NDN [10]	4
4	Processo de encaminhamento num nó NDN [10]	4
5	Exemplo de Vetor de Estado [5]	7
6	Exemplo de Vetor de Estado [8]	8
7	Fluxograma do funcionamento do SVSync [8]	10
8	Fluxograma do código.	15
9	Vetor de estado	16
10	Número de pacotes trocados.	17
11	Vetor de estado final.	18
12	Pivôs e provedor.	19
13	Número de pacotes trocados	19

Índice de Tabelas

Índice de Listagens

Siglas

CS	Content Store (pp. 4, 5)
FIB	Forwarding Information Base (pp. 4, 5)
MANETs	Redes móveis ad hoc (p. 2)
NDN	Named Data Network (pp. vi, vii, 1, 3–7, 9–11)
OBUs	On-Board Units (p. 2)
PIT	Pending Interest Table (pp. 4, 5)
RSUs	Roda Side Units (p. 2)
SVSync	State Vector Sync (pp. vi, vii, 1, 8–10)
V-NDN	Redes veiculares NDN (pp. vi, 5, 6)
V2I	Veículo para infraestrutura (p. 2)
V2N	Veículo para rede (p. 2)
V2P	Veículo para pessoa (p. 2)
V2V	Veículo para veículo (p. 2)
V2X	Veículo para todos (p. 2)
VANETs	Redes veiculares ad hoc (pp. vii, 1–3)

Capítulo 1

Introdução

1.1 Enquadramento e Motivação

O progresso das tecnologias de comunicações e das redes veiculares ad hoc (VANETs) têm produzido avanços no desenvolvimento de sistemas de transporte inteligentes. Neste contexto a troca de dados de forma eficiente e fiável entre veículos representa um grande desafio. O paradigma Named Data Network (NDN) aparece como uma alternativa às arquiteturas centradas em IP, pois privilegia uma comunicação com base em conteúdos característica de ambientes muito dinâmicos como os das VANETs. Uma parte essencial do NDN é a camada de sincronização, esta assegura a consistência e atualização dos dados, mas esses mecanismos de sincronização já existentes necessitam de ser adaptados ao cenário das redes veiculares, principalmente no que toca a latência, escalabilidade e à gestão de restrições geográficas e temporais. Esta dissertação tem como objetivo desenvolver uma camada de sincronização robusta, especificamente desenhada para o NDN aplicado às VANETs, contribuindo para a fiabilidade e o melhor desempenho das comunicações veiculares em contextos de sistemas de transporte.

1.2 Objetivos

Os objetivos desta dissertação passam por:

- Revisitar mecanismos já existentes de sincronização no NDN, neste caso o State Vector Sync SVSync e identificar algumas limitações nas redes veiculares.
- Propor uma solução para a sincronização adaptada às VANETs, incluindo as restrições geográficas e temporais
- Desenvolver algoritmos para a sincronização de dados nas redes veiculares.
- Implementar a solução proposta
- Avaliar a eficácia da solução proposta.

Capítulo 2

Estado da Arte

2.1 Redes Veiculares

As redes ad-hoc veiculares (VANETs) constituem uma variante especial das redes móveis ad-hoc (Redes móveis ad hoc (MANETs)), onde os veículos atuam como os nós móveis da rede comunicando entre si e com a infraestrutura da rede [4]. No contexto das VANETS as como Veículo para infraestrutura (V2I), as Veículo para veículo (V2V), as Veículo para pessoa (V2P), as Veículo para rede (V2N) e as Veículo para todos (V2X) como vemos na Figura 1.

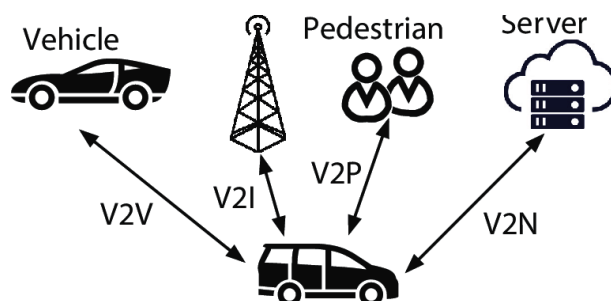


Figura 1: Mensagens das VANETs [1]

As VANETs possuem uma arquitetura composta por diferentes unidades, conhecidas como On-Board Units (OBUs) que são responsáveis pelo processamento local e as Road Side Units (RSUs) que fazem a conexão entre os veículos e a infraestrutura fixa. Esta infraestrutura distribuída permite que as redes VANETs suportem uma ampla gama de aplicações críticas, sobretudo em contextos urbanos, onde a densidade de veículos é elevada e a troca de informações precisa ser rápida e confiável. As VANETs apresentam características únicas, tais como uma topologia altamente dinâmica devido à elevada movimentação dos nós, requisitos de comunicação muito sensíveis ao tempo principalmente nas aplicações de segurança sendo que apesar de tudo os veículos estão cada vez mais equipados com mais altas capacidades de processamento e armazenamento. Estas particularidades, aliadas ao elevado custo, e falta de infraestrutura, tornam o desenvolvimento e implementação de protocolos de comunicação para VANETs um desafio muito grande. Para as VANETs existem 3 grandes tipos de aplicações as aplicações de segurança, de tráfego e de conforto, como vemos na Figura 2

Applications Type	Services Type	Local Interest	Space Validity	Time Validity
Safety Applications	Work Zone Warning	All vehicles	1 km	Until finishing work
	Car accident warning	All vehicles	500 m	30 Sec
	Dangerous road warning	All vehicles	500 m	10 Sec
Traffic Applications	Road congestion	All vehicles	5 km	30 min
	Traffic map	Subscriber	10 km	30 min
Comfort Applications	Multimedia file sharing	Subscriber	1 km	15 min
	Commercial advertisement	Subscriber	1-5 km	1-5 Days

Figura 2: Aplicações das VANETs [4]

2.1.1 Desafios das redes veiculares

Existem grandes desafios na implementação de VANETs, o facto da rede ser tão dinâmica visto que os nós estão sempre em movimento e por vezes a alta velocidade, leva a conexões curtas e por vezes falta de conectividade. A escalabilidade torna-se também uma preocupação, visto que a rede deve funcionar em cenários com variações extremas de nós, seja em regiões urbanas com grande tráfego ou regiões rurais que normalmente têm menos tráfego [2]. Outro aspeto crítico trata-se da segurança, visto que a natureza das VANETs torna-se bastante propensa a ataques, sendo que estes ataques podem ter consequências graves, não só em termos de informação mas como também na vida dos passageiros e pedestres[7].

2.2 Redes NDN

No início, quando as ideias centrais que fundamentaram a Internet começaram a ser desenvolvidas, a solução de comunicação oferecida pelo TCP/IP fosse única e revolucionária, o problema que ela resolveu era essencialmente o de conversas ponto a ponto entre duas entidades. Apesar de o TCP/IP ter sido concebido para conversos entre pontos finais de comunicação, acabou por superar todas as expectativas sendo também amplamente utilizado na distribuição de conteúdo, levando assim a Internet a passar a funcionar como uma rede de distribuição. Sendo assim tornou-se muito difícil resolver problemas, pois as redes de distribuição são mais complexas que as redes de comunicação. A natureza do IP está implementada nos seus datagramas, identificando apenas os pontos iniciais e finais de comunicação. O NDN propõe generalizar a arquitetura da Internet, removendo essa limitação, pois os nomes de um datagrama NDN passam a ser estruturados hierarquicamente, sendo que podem ser utilizados para identificar um pedaço de dados numa conversa ou um pedaço de conteúdo. Esta mudança, teoricamente simples, permite que se resolva uma alta gama de problemas de distribuição. A comunicação NDN é impulsionada pelos recetores, existindo apenas 2 pacotes: os pacotes de interesse e os pacotes de dados (Figura 3). O consumidor coloca o nome da parte dos dados desejada no pacote de interesse e envia-o para a rede, os routers usam o nome para encaminhar o interesse em direção aos produtores dos dados. Quando o interesse chega ao nó que possui os dados solicitados, esse nó retornará um pacote de dados que contém o nome e o conteúdo. Este pacote segue on caminho inverso ao caminho percorrido pelo

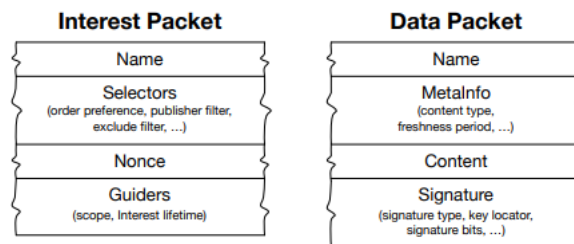


Figura 3: Pacotes das redes NDN [10]

pacote de interesse.

2.2.1 Funcionamento do NDN

Para realizar as funções de encaminhamento de interesses e dados, os routers NDN mantêm três estruturas: as Pending Interest Table (PIT), as Forwarding Information Base (FIB) e as Content Store (CS). A PIT é uma tabela que armazena todos os interesses que ainda não foram satisfeitos. Cada entrada registra o nome dos dados, juntamente com as interfaces de entrada e de saída. Quando chega um pacote de interesse, o router verifica a CS procurando os dados correspondentes, se existir o pacote de dados é logo retornado pela mesma interface pela qual chegou o interesse. Caso contrário, o router procura o nome na PIT, e se uma entrada correspondente existir, registra a interface de entrada na tabela. Se na PIT não houver nenhuma entrada correspondente ao nome, o router encaminha os dados ao produtor com base nas informações da sua FIB, bem como na estratégia de encaminhamento. Este processo pode ser visto na Figura 4.

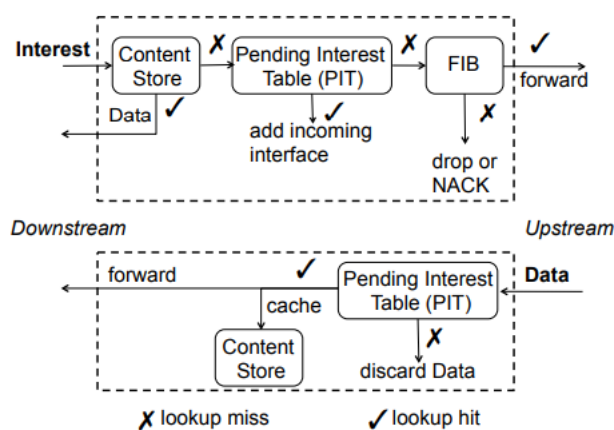


Figura 4: Processo de encaminhamento num nó NDN [10]

Quando um router recebe interesses para o mesmo nome de múltiplos nós ele vai apenas encaminhar apenas o primeiro em direção ao produtor de dados. A FIB é populada por um protocolo baseado no prefixo de nome e pode ter várias interfaces de saída para cada prefixo. A estratégia de encaminhamento pode decidir descartar um interesse em algumas situações, um exemplo, é se todos os links estiverem

congestionados ou em caso de suspeição de ataque. Para cada interesse, a estratégia de encaminhamento recupera a entrada correspondente ao prefixo mais longo da FIB e decide quando e para onde encaminhar os interesses. O CS é uma memória *cache* temporária de pacotes de dados recebidos pelo router. Como um pacote de dados NDN é independente de onde vem ou para onde vai, pode ser armazenado em cache para satisfazer futuros interesses. Quando chega um pacote de dados, o router NDN encontra a entrada na PIT e encaminha os dados para todas as interfaces nessa entrada, após isso remove essa PIT e armazena os dados na CS. Para grandes objetos de conteúdo que compreendem vários pacotes, os interesses desempenham um papel importantíssimo no controlo de fluxo como os *ACKs* do TCP. Nem os pacotes de interesse, nem os de dados transportam endereços, os routers encaminham os pacotes de dados com base nos nomes contidos nos pacotes e encaminham pacotes de dados para os consumidores com base nas informações de estado da PIT configuradas em pelos interesses em cada salto. Essa simetria na troca dos pacotes interesse/dados induz um loop de controlo *hop-by-hop*, e elimina a necessidade de qualquer noção de nós.

2.3 Vehicular Named Data Network (V-NDN)

No contexto dos veículos autónomos e redes ad-hoc veiculares (VANETs), os desafios inerentes à mobilidade e à intermitência de conectividade obstaculizam a manutenção de ligações ponto a ponto robustas [12]. Surge, assim, o paradigma V-DND, uma extensão do modelo Named Data Networking (NDN), que, em vez de depender de endereços IP, troca pacotes com base em nomes de dados, promovendo um encaminhamento orientado a conteúdos. A proposta V-DND assenta em três pilares:

- Encaminhamento baseado em nomes: o envio de Interest packets permite a descoberta eficiente de conteúdos e serviços disponíveis em nós móveis próximos, sem supor conectividade estável a um servidor central [9].
- Cache na rede: cada nó pode armazenar conteúdos recebidos, reforçando a disponibilidade de dados e reduzindo a latência em futuras solicitações.
- Resiliência face a falhas de rota: o comportamento reencaminhador NDN permite deteção adaptativa de falhas, evitando dependência de protocolos de routing tradicionais [13].

Já foi apresentado um protótipo V-NDN, seguido de testes experimentais que demonstraram o potencial do paradigma. Os resultados sugerem que as redes veiculares baseadas em V-NDN podem sustentar comunicações semipersistentes, adaptando-se dinamicamente à densidade e à mobilidade dos veículos. Sendo assim V-NDN apresenta grandes vantagens tais como a descoberta de conteúdo local, pois os veículos podem obter dados de outros nós próximos sem recorrer a infra-estrutura fixa. O encaminhamento adaptativo quando o sistema deteta falhas e redireciona automaticamente os pedidos por caminhos alternativos, sem necessidade de restabelecimento de sessão, é também uma vantagem a considerar e ainda a escalabilidade que tem haver com a utilização de cache distribuindo a carga de propagação de dados, melhorando a performance em ambientes densos [14]. Existem obviamente limitações tais como

a volatilidade dos nós (veículos que entram e saem do alcance) pode reduzir o aproveitamento de cache, a falta investigação aprofundada sobre o impacto da autenticação de dados, controlo de congestionamento e segurança frente a ataques do tipo Interest Flooding e a uniformidade na densidade de veículos e em curvas de tráfego necessita de ser explorada em cenários urbanos reais. O V-DND constitui uma extensão promissora do paradigma NDN ao domínio veicular, oferecendo resiliência, descentralização, e eficiência em redes móveis. Os resultados iniciais, embora positivos, apontam para a necessidade de investigação futura em cenários reais, com foco na segurança, controlo de contenda, e integração com serviços de transportes inteligentes.

2.3.1 Trabalhos em (V-NDN)

Num cenário de mobilidade elevada e redes intermitentes, os veículos conectados precisam de métodos robustos para troca de dados em tempo real. Como já foi dito o paradigma NDN emerge como uma alternativa viável aos modelos tradicionais IP-centrados, permitindo que a comunicação se foque no conteúdo e não nos nós de origem/destino. Esta arquitetura torna-se particularmente interessante quando se elimina a dependência de infraestrutura fixa, como as RSUs (Roadside Units), propondo um modelo puramente peer-to-peer entre veículos (V2V). Existem algumas abordagens, sendo que aqui vamos apresentar uma com recorrendo a RSU e outra, recorrendo apenas aos veículos. Nesta primeira abordagem, cada veículo age simultaneamente como produtor e consumidor de dados, permitindo a disseminação direta de conteúdos entre pares. A troca de dados baseia-se em nomes semânticos como por exemplo `ndn/highway/accident/segment42` e em estratégias adaptativas, garantindo que a informação mais relevante seja propagada eficientemente mesmo em condições de mobilidade elevada.[3] Este modelo elimina a necessidade de nós centrais, promovendo um sistema verdadeiramente distribuído, com caching local e interesses que fluem entre veículos adjacentes. Outra, abordagem, seria o uso do NDN em contextos de infraestrutura, onde os veículos interagem com semáforos inteligentes via pacotes Interest contendo dados sobre a sua posição, velocidade e destino[11]. Ainda que eficaz, esta solução pressupõe a existência de infraestruturas de apoio, o que limita a escalabilidade e aplicabilidade em zonas não urbanizadas. Ao comparar estas abordagens, torna-se evidente que os modelos V2V puros, como o V-NDN, oferecem maior resiliência, flexibilidade e independência de infraestrutura. São particularmente adequados para cenários onde a conectividade com RSUs não é garantida, como autoestradas, zonas rurais ou situações de emergência. Além disso, permitem a implementação de mecanismos como caching colaborativo e sincronização de dados em tempo real entre veículos, sem necessidade de coordenação externa.

2.4 Sincronização no NDN

O objetivo da sincronização no NDN é reconciliar as diferenças num conjunto de dados partilhado entre múltiplos participantes, ou seja, com o conhecimento de todos os dados disponíveis os utilizadores podem decidir quando e se recuperar dados em falta. À medida que novos dados são gerados pelos produtores,

é essencial que os membros sejam informados sobre alterações. Cada utilizador transmite o seu Estado para os outros participantes através de trocas de Interesse-Dados. Esta troca pode ser feita de duas forma, o produtor notificar ativamente sobre o seu estado, ou cada utilizador consultar periodicamente o grupo de sincronização. Uma abordagem para a reconciliação dos dados é o uso de vetores de estado. A ideia central é representar o estado de um conjunto de dados na forma de um vetor de versão, onde cada componente corresponde ao nome mais recente dos dados de cada produtor. Como um vetor de estado enumera diretamente o prefixo de cada produtor e o número de sequência dos dados mais recentes, as diferenças nos conjuntos de dados podem ser identificadas fazendo uma simples comparação entre os valores de cada prefixo. Para atualizar o seu Vetor de Estado, o nó adota o maior número de sequência disponível para cada produtor. O facto de a representação do estado baseada em Vetores de Estado permitir uma comparação direta do estado, permite que os nós identifiquem diretamente os dados em falta sem necessidade de trocas de mensagens adicionais. Esta característica torna a abordagem ideal para redes ad-hoc, onde grandes divergências de estado são bastante comuns. O tamanho do Vetor cresce proporcionalmente ao número de produtores cujos estados precisam ser trocados. Contudo, a listagem explícita dos estados de cada produtor permite a transmissão de Vetores de Estado parciais quando há restrições no tamanho dos pacotes. Como um Vetor de Estado enumera diretamente o prefixo do produtor e o número de sequência dos dados mais recentes para cada produtor num grupo de Sincronização, conforme mostrado na Figura 5, a diferença nos nomes dos conjuntos de dados pode ser identificada diretamente comparando o número de sequência de cada prefixo de produtor. Em seguida, um nó atualiza o seu Vetor de Estado adotando o valor mais alto do número de sequência de cada produtor no Vetor de Estado.

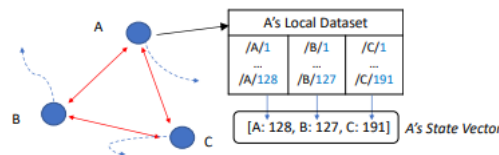


Figura 5: Exemplo de Vetor de Estado [5]

Como o NDN foi concebido para suportar comunicações assíncronas, exigir uma consistência perfeita das informações de associação entre todos os membros do grupo não é viável. Para contornar esta limitação, a sincronização por Vetores de Estado foi rapidamente sucedida por outro protocolo de sincronização, o DSSN. O DSSN trouxe uma alteração simples, mas crucial, permitindo a sincronização de conjuntos de dados em grupos de sensores que entram periodicamente em estados de inatividade. A principal mudança introduzida pelo DSSN foi a reformulação do vetor de estado, que passou de uma simples lista de números de sequência para uma lista de tuplas [prefixo-do-participante, número de sequência]. Com este novo formato, cada Interesse de Sincronização do DSSN transporta um vetor de estado que codifica diretamente o estado global do conjunto de dados partilhado. Esta abordagem garante que qualquer destinatário pode interpretar corretamente a mensagem DSSN, independentemente do nível de inconsistência de estado entre os participantes. O DSSN foi projetado para ambientes onde

a conectividade entre nós fixos é intermitente. A partir deste protocolo, surgiu o DDSN, uma extensão do DSSN otimizada para redes ad-hoc sem fio, caracterizadas por uma alta dinâmica no movimento dos nós. Para lidar com estas condições, o DDSN introduziu uma série de melhorias, incluindo:

- 1. Priorização de transmissão que permite determinar quais mensagens devem ser enviadas primeiro durante períodos de conectividade transitória.
- 2. Modo inativo que reduz o tráfego de sincronização e otimiza o consumo de energia quando não há outros nós dentro da área de comunicação.

Apesar da sua eficiência em ambientes altamente dinâmicos e disruptivos, o DDSN tem um desempenho inferior em redes bem conectadas, devido à sua ênfase exclusiva em cenários com conectividade instável. A experiência adquirida com os protocolos anteriores levou ao desenvolvimento do SVSync. Este protocolo herda a codificação do vetor de estado introduzida pelo DSSN, mas elimina funcionalidades específicas para redes de sensores e ambientes com elevada interrupção.

2.4.1 State Vector Sync SVSync

O SVSync adota uma estratégia de codificação de estado distinta. O seu design segue a convenção de nomeação sequencial dos dados e transporta diretamente a representação do namespace do conjunto de dados, codificada em pares [nome do produtor, número de sequência] (figura 6), dentro de cada Interesse Sync Multicast. A decisão de design do SVSync de incluir o namespace do conjunto de dados bruto em cada Interesse de Sincronização oferece uma vantagem significativa: qualquer nó que receba o Interesse compreende de imediato o estado do conjunto de dados, independentemente do seu próprio estado atual ou de eventuais mensagens perdidas anteriormente. No entanto, permitir que um Interesse recebido possa alterar o estado de um participante abre a porta a possíveis abusos. Para mitigar esse risco, o SVSync exige que todos os Interesses de Sincronização sejam assinados.

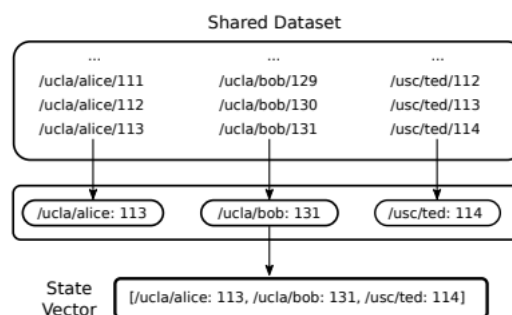


Figura 6: Exemplo de Vetor de Estado [8]

2.4.1.1 Funcionamento do SVSync

Um grupo de sincronização utiliza um prefixo multicast de grupo, garantindo que todos os participantes recebem as mensagens de sincronização. Cada membro do grupo possui também um prefixo de publicação específico, sob o qual os seus dados ficam acessíveis aos outros participantes. O SVSync opera com apenas um tipo de mensagem: o Interesse de Sincronização, utilizado para atualizar o estado do conjunto de dados. Estes Interesses de Sincronização são enviados para o prefixo multicast de grupo em duas situações:

- 1. Acionados por eventos, sempre que há uma alteração no estado do conjunto de dados, os participantes notificam imediatamente os restantes.
- 2. Envios periódicos, garantem uma visão consistente do conjunto de dados, mesmo em caso de perda de mensagens acionadas por eventos.

O primeiro caso é utilizado para disseminar rapidamente novas informações sobre o estado do conjunto de dados na rede, enquanto o segundo serve para garantir a consistência do estado do conjunto de dados do grupo, mesmo perante perdas de pacotes e desconexões temporárias. Os interesses de sincronização ativados por eventos garantem que o grupo se mantém atualizado sobre o estado do conjunto de dados partilhado. No entanto, o estado de um determinado membro pode ficar desatualizado relativamente aos restantes devido a diversas razões imprevisíveis, como perda de pacotes ou falhas temporárias de ligação. Para mitigar este problema, cada entidade de sincronização mantém um temporizador de interesse de sincronização para acionar periodicamente esses interesses, ajudando a manter os membros do grupo sincronizados. O funcionamento desse mecanismo é explicado na figura 7.

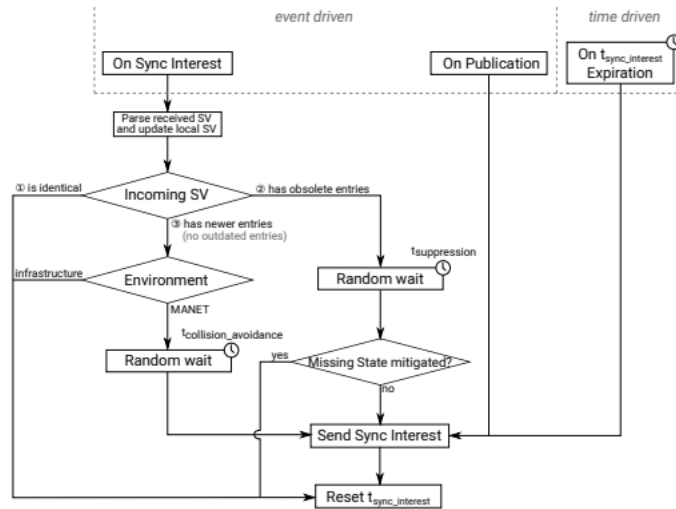


Figura 7: Fluxograma do funcionamento do SVSync [8]

Diferindo dos protocolos anteriores baseados em vetores, os interesses de sincronização SVSync são utilizados exclusivamente como notificações unidirecionais, sem desencadear pacotes de resposta. Todos os protocolos de sincronização utilizam interesses de sincronização para transmitir o estado do conjunto de dados, diferenciando-se apenas na forma como codificam esse estado. Os participantes de um grupo de sincronização transmitem interesses de sincronização multicast para todo o grupo. No entanto, solicitar notificações de mudança de estado através de interesses multicast levanta três desafios principais:

- 1. Como o momento da próxima alteração no estado do conjunto de dados é imprevisível, um interesse de sincronização que solicite uma resposta permanecerá pendente em todos os nós encaminhadores até expirar e ser renovado. Isto cria um estado PIT persistente para cada membro relativamente a todos os outros membros do grupo.
- 2. Um interesse multicast solicita uma resposta de vários produtores potenciais. Se múltiplos produtores responderem quase simultaneamente, devido ao princípio do NDN de um interesse um dado, apenas uma das respostas será entregue ao remetente do interesse.
- 3. Como consequência do ponto anterior, diferentes membros do grupo poderão receber atualizações distintas, levando a uma divergência do estado do conjunto de dados, que só convergirá após vários ciclos de troca de interesse-dado.

Para resolver estes problemas, em vez de solicitar respostas diretamente através de interesses de sincronização multicast, o SVSync utiliza-os apenas para permitir que cada participante notifique os restantes sobre o seu próprio estado do conjunto de dados.

2.4.2 Desafios das Redes Veiculares

A aplicação dos mecanismos de sincronização desenvolvidos para redes NDN em cenários veiculares enfrenta várias dificuldades intrínsecas. As VANETs caracterizam-se por uma topologia altamente dinâmica, em que os veículos se deslocam a velocidades elevadas e frequentemente entram e saem do alcance uns dos outros, o que resulta em ligações de curta duração e intermitência de conectividade. Essa volatilidade torna o processo de sincronização mais complexo do que em redes fixas ou mesmo em redes ad hoc com mobilidade reduzida. Além disso, a escalabilidade constitui um obstáculo relevante, pois em ambientes urbanos densos a quantidade de nós participantes cresce rapidamente, levando a um aumento considerável do número de mensagens de sincronização e ao risco de sobrecarga da rede. Outro aspecto crítico é a latência: muitas aplicações veiculares, sobretudo as associadas à segurança, requerem tempos de resposta de ordem de milissegundos, mas os protocolos de sincronização existentes nem sempre conseguem garantir convergência rápida em cenários de elevada mobilidade. A segurança e a resiliência também se revelam desafios, já que ataques como o flooding de interesses podem degradar significativamente o desempenho da rede e comprometer a fiabilidade da disseminação de dados. Estas limitações foram destacadas em diferentes estudos que analisam a adaptação do NDN às redes veiculares, confirmando a necessidade de soluções específicas que considerem as restrições geográficas e temporais, bem como os requisitos de baixa latência e alta escalabilidade.

Capítulo 3

Estratégia a utilizar

A escolha de uma arquitetura baseada em pivôs para a sincronização de dados em redes veiculares NDN justifica-se pela necessidade de reduzir o overhead de mensagens e melhorar a escalabilidade em cenários urbanos densos. Os mecanismos tradicionais de sincronização, como o SVSync, embora eficazes em redes ad hoc com número moderado de participantes, enfrentam dificuldades quando aplicados a situações em que dezenas de veículos necessitam de partilhar estados quase em simultâneo. Nestes casos, a disseminação em malha completa provoca um crescimento quase quadrático do número de interesses e estados na Pending Interest Table, levando a colisões, perdas e sobrecarga de processamento nos nós móveis. A introdução de pivôs permite alterar este padrão de comunicação, uma vez que os veículos sincronizam apenas com um representante da sua faixa e esse representante, por sua vez, troca vetores de estado apenas com outros pivôs. Desta forma, a complexidade do processo reduz-se significativamente, aproximando-se de uma relação linear com o número de nós e limitada por um número pequeno de trocas entre pivôs. Esta abordagem não apenas minimiza o tráfego mas também aproveita as características geográficas do cenário, pois em cruzamentos urbanos existe uma segmentação natural por faixas, o que facilita a eleição de nós agregadores.

A integração com o SVSync reforça a adequação da solução, dado que este protocolo já foi desenhado para suportar sincronização assíncrona eficiente com codificação explícita de vetores de estado em cada interesse. Ao delegar em pivôs a emissão dos anúncios de estado para fora da faixa, evita-se que múltiplos produtores gerem respostas concorrentes, problema reconhecido nos trabalhos de análise sobre protocolos de sincronização distribuída em NDN. Além disso, a arquitetura mantém compatibilidade com os princípios de funcionamento do NDN, explorando a multicast por nome e a cache de conteúdos na rede, enquanto reduz a latência na convergência de estados locais. Esta escolha também equilibra robustez e simplicidade: embora os pivôs se tornem potenciais pontos de falha, a utilização de mecanismos de eleição rápida e anúncios periódicos mitiga este risco, assegurando continuidade na disseminação de dados mesmo perante mobilidade elevada.

Em síntese, a adoção de pivôs como agregadores de sincronização em conjunto com o SVSync permite conciliar a necessidade de escalabilidade com a manutenção de baixa latência e fiabilidade na comunicação, explorando de forma eficiente as propriedades específicas das redes veiculares. Esta

estratégia surge alinhada com as limitações identificadas na literatura recente sobre sincronização em NDN e VANETs, sendo assim uma abordagem promissora para cenários urbanos de elevada densidade veicular

Capítulo 4

Implementação

Inicialmente, são definidos os parâmetros de entrada que configuram intervalos de publicação rápidos e lentos, a quantidade de publicações recentes e aleatórias que devem ser consideradas pelo mecanismo de sincronização SVS, a taxa de perda de pacotes no canal e a possibilidade de fragmentação de pacotes. Em seguida, são configurados os dispositivos de rede e canais com um modelo de erro de taxa, largura de banda de 50 Mbps e atraso fixo de 5 ms. A topologia adotada é um grid de 5x5 nós interconectados por enlaces ponto a ponto. Sobre essa topologia, é instalada a pilha NDN em todos os nós, bem como tracers que permitem registrar estatísticas de tráfego, atraso de aplicações e estado do Content Store. Cada nó da grelha, exceto o nó central (2,2), recebe uma aplicação de chat baseada em SVS. Cada aplicação tem um prefixo próprio e uma cadência de publicação que pode ser rápida ou lenta dependendo da paridade da posição do nó. Esses nós são também registrados como origens de conteúdo para que possam ser alcançados através das rotas calculadas globalmente. Paralelamente, a classe HierarchicalSyncMobilityManager é responsável por orquestrar a mobilidade lógica dos nós, que consiste em movê-los ao ponto central. Cada nó possui metadados que incluem sua posição, um identificador lógico, a versão inicial dos dados e se ele atua como pivô ou provedor de dados. Os pivôs são definidos como os nós localizados na última coluna, e o provedor de dados é selecionado como aquele com a versão inicial mais alta. Quando todos os 24 nós periféricos atingem o centro, a simulação registra o instante de início da sincronização e quando termina o tempo de término é registrado e a duração total do processo é calculada. Em termos funcionais, todos os nós atualizam sua versão de dados para a maior versão observada na rede, simulando o efeito da convergência proporcionada pelo SVS. O estado final de cada nó é então impresso e, para enriquecer a análise. Para simplificar esta análise podemos visualizar na figura 8 um fluxograma do código feito.

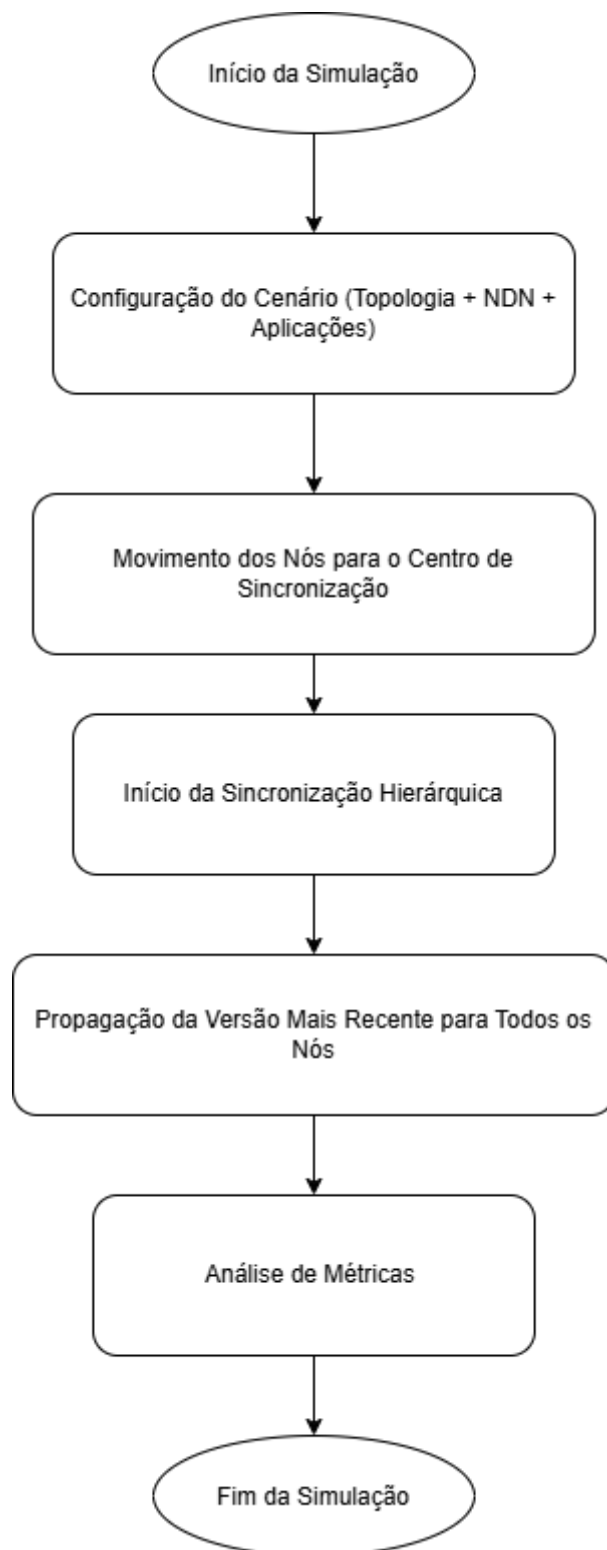


Figura 8: Fluxograma do código.

Capítulo 5

Resultados

Para demonstrar a diferença entre a sincronização normal num cruzamento, e a sincronização de com pivôs vão ser apresentados os resultados das duas implementações. Para simular os resultados foram utilizados 24 nós. Para começar apresentam-se os resultados da simulação sem pivô. Na figura 9 temos o vetor de estado final da implementação. Como podemos ver tudo correu bem pois a versão final corresponde à versão mais recente em todos os nós da simulação.

```
=== RESULTADO FINAL DA SINCRONIZACAO ===
Node-0-0 (versao inicial: 10) -> Versao final: 15
Node-0-1 (versao inicial: 12) -> Versao final: 15
Node-0-2 (versao inicial: 5) -> Versao final: 15
Node-0-3 (versao inicial: 12) -> Versao final: 15
Node-0-4 (versao inicial: 1) -> Versao final: 15
Node-1-0 (versao inicial: 10) -> Versao final: 15
Node-1-1 (versao inicial: 8) -> Versao final: 15
Node-1-2 (versao inicial: 10) -> Versao final: 15
Node-1-3 (versao inicial: 5) -> Versao final: 15
Node-1-4 (versao inicial: 2) -> Versao final: 15
Node-2-0 (versao inicial: 14) -> Versao final: 15
Node-2-1 (versao inicial: 5) -> Versao final: 15
Node-2-3 (versao inicial: 14) -> Versao final: 15
Node-2-4 (versao inicial: 14) -> Versao final: 15
Node-3-0 (versao inicial: 7) -> Versao final: 15
Node-3-1 (versao inicial: 11) -> Versao final: 15
Node-3-2 (versao inicial: 6) -> Versao final: 15
Node-3-3 (versao inicial: 13) -> Versao final: 15
Node-3-4 (versao inicial: 5) -> Versao final: 15
Node-4-0 (versao inicial: 15) -> Versao final: 15
Node-4-1 (versao inicial: 4) -> Versao final: 15
Node-4-2 (versao inicial: 5) -> Versao final: 15
Node-4-3 (versao inicial: 8) -> Versao final: 15
Node-4-4 (versao inicial: 3) -> Versao final: 15
```

Figura 9: Vetor de estado

De seguida na figura 10 podemos ver o número de pacotes que foram trocados, e isso vai ser muito importante para as conclusões finais desta secção. Mas, por alto podemos ver um elevado número de

pacotes, que se deve a colisões e perdas durante a sincronização, visto que o processo neste caso é bastante mais desorganizado.

```
===== ANÁLISE DE LATÊNCIA =====  
Latência: 6.0000s  
  
===== ANÁLISE DE TRÁFEGO E PERDA DE PACOTES =====  
Interesses Transmítidos (Total): 89255  
Interesses Recebidos (Total): 24268  
Dados Recebidos (Total): 138  
  
Interesses Perdidos: 64987  
  
Total de pacotes SVS: 113662  
Taxa de pacotes/segundo: 18943.66
```

Figura 10: Número de pacotes trocados.

Passando agora para os resultados da simulação que implementa a solução proposta. Começando também com o vetor de estado final representado na figura 11, podemos também comprovar que realmente a troca de dados está a acontecer de forma correta, pois a versão final corresponde novamente à mais recente.

Na figura 12 podemos ver quem foi o provedor dos dados mais recentes, e também a comprovar a existência de um pivô por cada faixa, que irá ser o ponto fulcral da diferença esperada entre as 2 soluções.

Na figura 13 podemos ver o valor, que continua algo alto por ainda existirem colisões e perdas.

```
=== RESULTADO FINAL DA SINCRONIZAÇÃO ===  
Node-0-0 (versão inicial: 15) → Versão final: 15  
Node-0-1 (versão inicial: 11) → Versão final: 15  
Node-0-2 (versão inicial: 11) → Versão final: 15  
Node-0-3 (versão inicial: 13) → Versão final: 15  
Node-0-4 (versão inicial: 7) → Versão final: 15  
Node-1-0 (versão inicial: 8) → Versão final: 15  
Node-1-1 (versão inicial: 13) → Versão final: 15  
Node-1-2 (versão inicial: 7) → Versão final: 15  
Node-1-3 (versão inicial: 1) → Versão final: 15  
Node-1-4 (versão inicial: 10) → Versão final: 15  
Node-2-0 (versão inicial: 10) → Versão final: 15  
Node-2-1 (versão inicial: 13) → Versão final: 15  
Node-2-3 (versão inicial: 2) → Versão final: 15  
Node-2-4 (versão inicial: 12) → Versão final: 15  
Node-3-0 (versão inicial: 8) → Versão final: 15  
Node-3-1 (versão inicial: 11) → Versão final: 15  
Node-3-2 (versão inicial: 8) → Versão final: 15  
Node-3-3 (versão inicial: 7) → Versão final: 15  
Node-3-4 (versão inicial: 2) → Versão final: 15  
Node-4-0 (versão inicial: 10) → Versão final: 15  
Node-4-1 (versão inicial: 2) → Versão final: 15  
Node-4-2 (versão inicial: 6) → Versão final: 15  
Node-4-3 (versão inicial: 10) → Versão final: 15  
Node-4-4 (versão inicial: 12) → Versão final: 15
```

Figura 11: Vetor de estado final.

```
=== CONFIGURAÇÃO DE SINCRONIZAÇÃO HIERÁRQUICA ===
Nós movem-se para o centro e a sincronização é medida por tracers.
Node-0-0 configurado em (0,0) com versão inicial: 15
Node-0-1 configurado em (0,1) com versão inicial: 11
Node-0-2 configurado em (0,2) com versão inicial: 11
Node-0-3 configurado em (0,3) com versão inicial: 13
Node-0-4 configurado em (0,4) (PIVÔ) com versão inicial: 7
Node-1-0 configurado em (1,0) com versão inicial: 8
Node-1-1 configurado em (1,1) com versão inicial: 13
Node-1-2 configurado em (1,2) com versão inicial: 7
Node-1-3 configurado em (1,3) com versão inicial: 1
Node-1-4 configurado em (1,4) (PIVÔ) com versão inicial: 10
Node-2-0 configurado em (2,0) com versão inicial: 10
Node-2-1 configurado em (2,1) com versão inicial: 13
Node-2-3 configurado em (2,3) com versão inicial: 2
Node-2-4 configurado em (2,4) (PIVÔ) com versão inicial: 12
Node-3-0 configurado em (3,0) com versão inicial: 8
Node-3-1 configurado em (3,1) com versão inicial: 11
Node-3-2 configurado em (3,2) com versão inicial: 8
Node-3-3 configurado em (3,3) com versão inicial: 7
Node-3-4 configurado em (3,4) (PIVÔ) com versão inicial: 2
Node-4-0 configurado em (4,0) com versão inicial: 10
Node-4-1 configurado em (4,1) com versão inicial: 2
Node-4-2 configurado em (4,2) com versão inicial: 6
Node-4-3 configurado em (4,3) com versão inicial: 10
Node-4-4 configurado em (4,4) (PIVÔ) com versão inicial: 12
O provedor de dados será: Node-0-0 com a versão inicial mais alta: 15
```

Figura 12: Pivôs e provedor.

```
-----
=== ANÁLISE DE LATÊNCIA ===
-----
Latência: 2.0000s

-----
=== ANÁLISE DE TRÁFEGO E PERDA DE PACOTES ===
-----
Interesses Transmítidos (Total): 29887
Interesses Recebidos (Total): 8417
Dados Recebidos (Total): 44
-----
Interesses Perdidos: 21469
-----
Total de pacotes SVS: 38349
Taxa de pacotes/segundo: 19174.37
```

Figura 13: Número de pacotes trocados

Capítulo 6

Conclusão e trabalho futuro

Concluindo como podemos ver a solução proposta acaba por ser uma ajuda bastante grande na diminuição dos pacotes trocados, permitindo assim uma maior escalabilidade da rede. A primeira solução, sem pivôs acaba por se tornar mais confusa e gerar um tráfego elevadíssimo devido a muitas colisões e perdas de interesses e dados no caminho, visto que os 24 nós estão a enviar todos interesses entre si o que causa uma sobrecarga. Na segunda solução com pivôs esse valor diminui drasticamente, e isso acontece pois no centro apenas os 4 pivôs trocam dados entre si, os outros nós estão à espera que termine esta "fase" de sincronização entre pivôs para depois enviarem novamente interesses. Ainda assim o valor é alto pois existem algumas colisões e perdas no caminho. Como podemos observar a solução de sincronização com pivôs acaba por se tornar mais eficiente e escalável que a solução sem pivôs, pelo que comprovamos que realmente a solução proposta no capítulo 3, é positiva e ajuda a resolver muitos dos problemas da sincronização nas redes veiculares, tais como a escalabilidade e a latência.

Apesar das vantagens da solução proposta, a sua aplicação em cenários veiculares apresenta algumas limitações que importa reconhecer. A elevada mobilidade dos veículos e a consequente intermitência de conectividade podem comprometer a eficácia do processo de sincronização, já que os nós podem abandonar o cruzamento ou mudar de faixa antes de concluírem a atualização do seu vetor de estado. Esta característica torna a convergência temporalmente dependente e sujeita a falhas de consistência momentânea. Outro desafio relevante é a elevada taxa de colisões e perdas de pacotes no meio sem fios, particularmente em ambientes urbanos densos, onde a contenção no canal de comunicação aumenta de forma significativa. Mesmo que a arquitetura com pivôs reduza o número de mensagens globais, a qualidade do canal pode limitar a fiabilidade da sincronização. Adicionalmente, a escolha de pivôs introduz um risco de pontos únicos de falha: se o pivô de uma faixa ficar indisponível, o processo de disseminação pode ser temporariamente interrompido, exigindo mecanismos rápidos de eleição de substitutos. Também se deve considerar o custo computacional da assinatura digital dos interesses de sincronização exigida pelo SVSync, que pode tornar-se pesado para unidades embarcadas com recursos limitados. Por outro lado, a escalabilidade global do sistema continua a depender da forma como os vetores de estado são codificados, uma vez que o tamanho destes cresce com o número de produtores, o que pode gerar overhead em cenários de tráfego intenso.

Bibliografia

- [1] FIGURE 1. Four types of V2X application support in 3GPP (i.e., V2V,... en. url: https://www.researchgate.net/figure/Four-types-of-V2X-application-support-in-3GPP-ie-V2V-V2P-V2N-and-V2I_fig1_321088744 (acedido em 2025-04-22) (ver p. 2).
- [2] M. Gerla et al. «Internet of vehicles: From intelligent grid to autonomous cars and vehicular clouds». Em: *2014 IEEE World Forum on Internet of Things (WF-IoT)*. 2014-03, pp. 241–246. doi: [10.1109/WF-IoT.2014.6803166](https://doi.org/10.1109/WF-IoT.2014.6803166). url: <https://ieeexplore.ieee.org/abstract/document/6803166> (acedido em 2025-07-01) (ver p. 3).
- [3] G. Grassi et al. *Vehicular Inter-Networking via Named Data*. arXiv:1310.5980 [cs]. 2013-10. doi: [10.48550/arXiv.1310.5980](https://doi.org/10.48550/arXiv.1310.5980). url: <http://arxiv.org/abs/1310.5980> (acedido em 2025-06-30) (ver p. 6).
- [4] H. Khelifi et al. «Named Data Networking in Vehicular Ad Hoc Networks: State-of-the-Art and Challenges». Em: *IEEE Communications Surveys & Tutorials* 22.1 (2020), pp. 320–351. issn: 1553-877X. doi: [10.1109/COMST.2019.2894816](https://doi.org/10.1109/COMST.2019.2894816). url: <https://ieeexplore.ieee.org/abstract/document/8624354> (acedido em 2025-04-22) (ver pp. 2, 3).
- [5] T. Li et al. «A Brief Introduction to NDN Dataset Synchronization (NDN Sync)». en. Em: *MILCOM 2018 - 2018 IEEE Military Communications Conference (MILCOM)*. Los Angeles, CA: IEEE, 2018-10, pp. 612–618. isbn: 978-1-5386-7185-6. doi: [10.1109/MILCOM.2018.8599772](https://doi.org/10.1109/MILCOM.2018.8599772). url: <https://ieeexplore.ieee.org/document/8599772/> (acedido em 2025-03-24) (ver p. 7).
- [6] J. M. Lourenço. *The NOVAthesis L^AT_EX Template User's Manual*. NOVA University Lisbon. 2021. url: <https://github.com/joaomlourenco/novathesis/raw/main/template.pdf> (ver p. ii).
- [7] N. Lu et al. «Connected Vehicles: Solutions and Challenges». Em: *IEEE Internet of Things Journal* 1.4 (2014-08), pp. 289–299. issn: 2327-4662. doi: [10.1109/JIOT.2014.2327587](https://doi.org/10.1109/JIOT.2014.2327587). url: <https://ieeexplore.ieee.org/document/6823640> (acedido em 2025-07-01) (ver p. 3).
- [8] P. Moll et al. «A Brief Introduction to State Vector Sync». en. Em: () (ver pp. 8, 10).

- [9] C. Papadopoulos, S. Shannigrahi e A. Afanaseyv. «In-vehicle networking with NDN». Em: *Proceedings of the 8th ACM Conference on Information-Centric Networking*. ICN '21. New York, NY, USA: Association for Computing Machinery, 2021-09, pp. 127–129. isbn: 978-1-4503-8460-5. doi: [10.1145/3460417.3483374](https://doi.org/10.1145/3460417.3483374). url: <https://doi.org/10.1145/3460417.3483374> (acedido em 2025-06-23) (ver p. 5).
- [10] C. Papadopoulos, L. Wang e B. Zhang. «Lixia Zhang Alexander Afanasyev Jeffrey Burke Van Jacobson». en. Em: *ACM SIGCOMM Computer Communication Review* 44.3 (2014) (ver p. 4).
- [11] M. Al-qutwani e X. Wang. «Smart Traffic Lights over Vehicular Named Data Networking». en. Em: *Information* 10.3 (2019-03). Number: 3 Publisher: Multidisciplinary Digital Publishing Institute, p. 83. issn: 2078-2489. doi: [10.3390/info10030083](https://doi.org/10.3390/info10030083). url: <https://www.mdpi.com/2078-2489/10/3/83> (acedido em 2025-06-30) (ver p. 6).
- [12] M. A. Rahman e B. Zhang. «On Data-centric Forwarding in Mobile Ad-hoc Networks: Baseline Design and Simulation Analysis». en. Em: *2021 International Conference on Computer Communications and Networks (ICCCN)*. Athens, Greece: IEEE, 2021-07, pp. 1–9. isbn: 978-1-66541-278-0. doi: [10.1109/ICCCN52240.2021.9522163](https://doi.org/10.1109/ICCCN52240.2021.9522163). url: <https://ieeexplore.ieee.org/document/9522163/> (acedido em 2025-06-23) (ver p. 5).
- [13] M. A. Rahman e B. Zhang. «On the Analysis of Adaptive-Rate Applications in Data-Centric Wireless Ad-Hoc Networks». en. Em: *2021 IEEE 46th Conference on Local Computer Networks (LCN)*. Edmonton, AB, Canada: IEEE, 2021-10, pp. 503–510. isbn: 978-1-66541-886-7. doi: [10.1109/LCN52139.2021.9524982](https://doi.org/10.1109/LCN52139.2021.9524982). url: <https://ieeexplore.ieee.org/document/9524982/> (acedido em 2025-06-23) (ver p. 5).
- [14] T. Yu et al. «Names to Rule Them All: Unifying Vehicular Networking via Named Secured Data». en. Em: () (ver p. 5).